

Véridis

Proyecto Intermodular del Ciclo Superior de
Desarrollo de Aplicaciones Multiplataforma

Desarrollado por Lucía Grau González

Índice

1. Análisis del problema.....	3
1.1 Introducción.....	3
1.2 Objetivos Principales.....	3
1.3 Funciones y rendimientos deseados.....	4
1.4 Planteamiento y evaluación de diversas soluciones.....	5
1.5 Justificación de la solución elegida.....	7
1.6 Modelado de la solución.....	7
2. Diseño e implantación del proyecto.....	9
2.1 Plan de empresa.....	9
2.2 Arquitectura de la aplicación.....	10
2.3 Modelo de funcionamiento.....	11
2.4 Diseño de la base de datos.....	11
2.5 Mapa de navegación de la aplicación.....	15
3. Fase de pruebas.....	18
3.1 Objetivos de las pruebas.....	19
3.2 Tipos de pruebas realizadas.....	19
3.3 Batería de pruebas.....	20
3.4 Criterios de aceptación.....	24
3.5 Resultados de la fase de pruebas.....	24
4. Documentación de la aplicación.....	27
4.1 Introducción a la aplicación.....	27
4.2 Manual de instalación.....	28
4.3 Manual de administración.....	30
5. Conclusiones finales.....	33
5.1 Propuestas de modificación o ampliaciones futuras.....	34
6. Bibliografía.....	34

1. ANÁLISIS DEL PROBLEMA

1.1 INTRODUCCIÓN

Véridis es una aplicación móvil orientada al seguimiento y cuidado de plantas domésticas. El proyecto nace como respuesta a una necesidad habitual entre personas que poseen varias plantas en casa: organizar sus cuidados, recordar tareas importantes y disponer de información clara sobre las necesidades de cada especie.

Muchas personas adquieren plantas sin conocer con precisión sus requisitos de riego, luz, humedad, temperatura o mantenimiento. Esto puede provocar olvidos, cuidados inadecuados o dificultad para detectar problemas frecuentes como falta de agua, exceso de riego, mala ubicación o deterioro general de la planta. Además, cuando el usuario tiene un catálogo amplio de plantas, la gestión manual de todas sus rutinas se vuelve poco práctica.

La solución propuesta consiste en una aplicación móvil que permite registrar las plantas del usuario, consultar información botánica básica, organizar ubicaciones dentro del hogar, establecer recordatorios de cuidado y consultar recomendaciones apoyadas en datos estructurados y condiciones climáticas. De esta forma, Véridis busca convertirse en una herramienta útil, visual y accesible para mejorar la relación entre el usuario y el cuidado diario de sus plantas.

El sistema se plantea como una aplicación multiplataforma desarrollada con React Native y Expo para la parte móvil, un backend basado en Spring Boot y JPA, y una base de datos relacional MySQL. La comunicación entre la aplicación y la base de datos se realizará mediante una API REST, lo que permite separar correctamente la interfaz de usuario, la lógica del servidor y la persistencia de datos.

1.2 OBJETIVOS PRINCIPALES

El objetivo principal del proyecto es desarrollar una aplicación móvil que permita a los usuarios gestionar y cuidar sus plantas domésticas de forma organizada, personalizada y sencilla.

Para conseguirlo, el sistema deberá permitir el registro de usuarios, la creación de un catálogo personal de plantas, la consulta de información sobre especies vegetales, la programación de recordatorios y el seguimiento de las acciones de cuidado realizadas sobre cada planta.

Además, la aplicación pretende mejorar la experiencia del usuario mediante una interfaz cuidada, clara y visual, evitando que el cuidado de plantas dependa únicamente de notas externas, memoria personal o información dispersa en internet.

Como objetivos principales del sistema se establecen los siguientes:

- Permitir que el usuario registre y gestione sus propias plantas.
- Ofrecer información estructurada sobre especies, familias botánicas y cuidados recomendados.
- Facilitar la organización de plantas por ubicaciones dentro del hogar.
- Registrar un historial de cuidados realizados.
- Generar recordatorios para tareas como riego, abonado, poda o trasplante.
- Integrar información climática para apoyar recomendaciones de cuidado.
- Diseñar una aplicación visualmente atractiva, intuitiva y adecuada para dispositivos móviles.
- Construir una arquitectura clara basada en aplicación móvil, API REST y base de datos relacional.

1.3 FUNCIONES Y RENDIMIENTOS DESEADOS

Una vez implementado, Véridis ofrecerá servicios centrados en la gestión del cuidado de plantas domésticas. La aplicación permitirá crear una cuenta, iniciar sesión y gestionar un perfil personal con las plantas del usuario.

El sistema contará con una base de datos de plantas base organizada por familias botánicas, con información sobre especies, cuidados, dificultad, toxicidad, luz, riego, temperatura, humedad y consejos. A partir de estas plantas, el usuario podrá registrar sus propios ejemplares, añadiendo nombre personalizado, fotografía, ubicación, estado de salud, notas y fechas de cuidados.

Además, la aplicación permitirá guardar un historial de acciones realizadas, como riegos, abonados, podas o trasplantes, y gestionar recordatorios asociados a cada planta. También se contempla el uso de información meteorológica para ofrecer recomendaciones más contextualizadas.

En cuanto al rendimiento, Véridis deberá funcionar de forma fluida en dispositivos móviles, cargar la información de manera eficiente mediante una API REST y mantener los datos organizados en una base de datos MySQL. En conjunto, el sistema busca ofrecer una experiencia sencilla, ordenada y fiable para mejorar el cuidado diario de las plantas.

1.4 PLANTEAMIENTO Y EVALUACIÓN DE DIVERSAS SOLUCIONES

Para resolver el problema de Véridis se han valorado varias alternativas. La primera opción sería utilizar herramientas manuales, como notas, calendarios u hojas de cálculo. Aunque es una solución sencilla, depende demasiado de la organización del usuario y no permite estructurar bien la información ni integrar recordatorios, historial o recomendaciones.

Otra posibilidad sería crear una aplicación web responsive, accesible desde navegador. Esta opción permitiría usar el sistema desde distintos dispositivos y facilitaría las actualizaciones, pero no se adapta tan bien al uso diario desde el móvil, especialmente para notificaciones, fotografías y tareas rápidas.

También se ha considerado una aplicación móvil con almacenamiento local. Esta alternativa sería rápida y sencilla, pero tendría limitaciones importantes, como la pérdida de datos al cambiar de dispositivo, la falta de sincronización y una menor adecuación a los requisitos técnicos del proyecto.

Finalmente, se plantea una aplicación móvil conectada a una API REST y a una base de datos MySQL. Esta opción permite separar la interfaz, la lógica del servidor y los datos, además de gestionar usuarios, plantas, recordatorios e historial de forma centralizada. Aunque requiere más desarrollo, se considera la solución más completa, escalable y adecuada para los objetivos del proyecto.

Evaluación comparativa de las soluciones

Tras analizar las distintas alternativas, se concluye que la opción más conveniente es desarrollar una aplicación móvil conectada a una API REST y a una base de datos MySQL. Esta solución permite cubrir las necesidades principales del usuario, mantener una estructura técnica sólida y construir un sistema ampliable en el futuro.

Solución	Ventajas	Inconvenientes	Valoración
Gestión manual	Sencilla, rápida y sin desarrollo técnico	Poco estructurada, dependiente del usuario y limitada	No adecuada para el alcance del proyecto
Aplicación web	Accesible desde el navegador y fácil de	Menor integración con funciones móviles	Viable, pero menos adecuada

	actualizar		
App móvil con almacenamiento local	Rápida y funcional sin usar un servidor	Riesgo de pérdida de datos y poca escalabilidad	Útil solo para prototipos simples
App móvil con API REST y MYSQL	Completa, escalable y adecuada al uso móvil	Mayor complejidad de desarrollo	Solución más adecuada

1.5 JUSTIFICACIÓN DE LA SOLUCIÓN ELEGIDA

Se ha elegido desarrollar Véridis como una aplicación móvil multiplataforma conectada a una API REST y a una base de datos MySQL, ya que es la opción que mejor se adapta al uso diario de la aplicación. Al estar pensada para consultar plantas, registrar cuidados y recibir recordatorios desde el móvil, una app ofrece una experiencia más cómoda que una solución web.

React Native con Expo permite crear la aplicación para Android e iOS con una sola base de código y facilita el uso de funciones del dispositivo, como notificaciones e imágenes. La API REST separa la aplicación móvil del servidor y de la base de datos, mejorando la organización, la seguridad y la posibilidad de ampliar el proyecto.

MySQL se considera adecuado porque Véridis gestiona datos estructurados y relacionados, como usuarios, plantas, familias botánicas, recordatorios, historial de cuidados y ubicaciones. Además, se ha decidido no incluir funciones complejas de inteligencia artificial, ya que el objetivo principal es crear una solución realista, estable y centrada en el seguimiento del cuidado de plantas domésticas.

Por tanto, la solución elegida cumple los objetivos del proyecto y se ajusta a una arquitectura profesional basada en cliente móvil, backend y base de datos.

1.6 MODELADO DE LA SOLUCIÓN

La solución propuesta para Véridis se basa en una arquitectura cliente-servidor formada por tres partes principales: una aplicación móvil, una API REST y una base de datos MySQL.

La aplicación móvil actuará como cliente y será el punto de interacción con el usuario. Desde ella se podrán realizar acciones como registrarse, iniciar sesión, consultar plantas, añadir plantas propias, revisar recordatorios, registrar cuidados y acceder a la información de cada ejemplar.

La API REST funcionará como intermediaria entre la aplicación móvil y la base de datos. Su función será recibir las peticiones del cliente, validar los datos, aplicar la lógica necesaria y devolver una respuesta adecuada. De esta forma, la aplicación no accederá directamente a la base de datos, lo que mejora la organización y seguridad del sistema.

La base de datos MySQL almacenará la información persistente del proyecto. En ella se guardarán datos relacionados con usuarios, plantas base, familias botánicas, plantas del usuario, ubicaciones, recordatorios e historial de cuidados. El diseño detallado de sus tablas y relaciones se desarrollará en el apartado correspondiente al diseño de la base de datos.

El funcionamiento general del sistema seguirá este esquema:

Aplicación móvil → API REST → Base de datos MySQL

Esta estructura permite separar responsabilidades, facilitar el mantenimiento del proyecto y dejar preparada la aplicación para futuras ampliaciones.

1.6.1 Recursos humanos

El proyecto ha sido desarrollado por la alumna responsable, **Lucía Grau**, encargada del análisis, diseño, desarrollo, pruebas y documentación de la aplicación.

Además, se han utilizado diferentes usuarios de prueba para obtener feedback durante el desarrollo. Sus opiniones han servido para valorar la claridad de la interfaz, la facilidad de uso y la comprensión de las funcionalidades principales de Véridis.

1.6.2 Recursos Hardware

Para desarrollar el proyecto se utilizará un ordenador personal capaz de ejecutar el entorno de programación, el backend, la base de datos y las herramientas de prueba. También será necesario un dispositivo móvil físico o un emulador para comprobar el funcionamiento de la aplicación. En caso de despliegue, podrá emplearse un servidor remoto para alojar la API REST y la base de datos.

1.6.3 Recursos Software

El proyecto utilizará React Native con Expo para el desarrollo de la aplicación móvil, Node.js y Express para el backend, y MySQL como sistema de base de datos. Para la comunicación entre cliente y servidor se utilizarán peticiones HTTP a una API REST con respuestas en formato JSON.

Además, podrán emplearse herramientas como Visual Studio Code para el desarrollo, MySQL Workbench o phpMyAdmin para gestionar la base de datos, Git y GitHub para el control de versiones, Expo Notifications para los recordatorios, Open-Meteo para la información meteorológica y Figma o Canva para el diseño visual y prototipado de la interfaz.

2. DISEÑO E IMPLANTACIÓN DEL PROYECTO

En este apartado se desarrolla la parte técnica del proyecto, siguiendo los criterios establecidos para una aplicación multiplataforma. Se incluye el plan de empresa, la arquitectura de la aplicación, los tipos de usuarios, el mapa de navegación, el diseño de la base de datos y la relación del proyecto con los módulos del ciclo.

El proyecto se plantea como una aplicación móvil conectada a una API REST y a una base de datos MySQL. Esta estructura permite separar la interfaz, la lógica del servidor y la persistencia de datos, facilitando el mantenimiento y la ampliación futura del sistema.

2.1 PLAN DE EMPRESA

Para contextualizar el proyecto, se plantea una empresa ficticia llamada **Véridis Studio**, dedicada al desarrollo de soluciones digitales relacionadas con el cuidado de plantas domésticas.

La empresa centra su actividad en crear herramientas móviles sencillas, visuales y útiles para usuarios que desean organizar mejor sus rutinas de cuidado vegetal. Su primer producto es **Véridis**, una aplicación orientada a registrar plantas, consultar cuidados, crear recordatorios y realizar seguimiento de acciones.

El modelo de funcionamiento se basa en una aplicación gratuita para el usuario final. La infraestructura técnica, formada por el backend y la base de datos, se aloja en la nube, permitiendo que el usuario solo necesite instalar la app en su dispositivo móvil.

2.1.1 Actividad principal

La actividad principal de Véridis Studio es el diseño, desarrollo y mantenimiento de una aplicación móvil multiplataforma. La empresa se encarga de definir la estructura funcional, diseñar la interfaz, implementar el backend, gestionar la base de datos y mantener actualizada la información general del sistema.

2.1.2 Público objetivo

El público objetivo está formado por personas que tienen plantas en casa y necesitan una forma sencilla de organizar sus cuidados. La aplicación se dirige tanto a usuarios principiantes como a personas con varias plantas que desean llevar un control más ordenado.

2.1.3 Necesidad cubierta

El proyecto cubre la necesidad de centralizar en una sola herramienta la información y las rutinas de cuidado de cada planta. De esta forma, se evitan olvidos, desorganización y dependencia de notas externas o información dispersa.

2.2 ARQUITECTURA DE LA APLICACIÓN

La arquitectura del sistema se basa en un modelo cliente-servidor compuesto por tres partes principales:

Aplicación móvil > API REST > Base de datos

La aplicación móvil será desarrollada con **React Native y Expo**. Esta parte se encargará de mostrar la interfaz, permitir la navegación entre pantallas y recoger las acciones del usuario.

La API REST será desarrollada con **Spring Boot y JPA**. Su función será recibir las peticiones de la app, validar los datos, aplicar la lógica necesaria y comunicarse con la base de datos.

La base de datos será **MySQL** y estará alojada en la nube. En ella se almacenará la información persistente del sistema: usuarios, plantas base, plantas personales, familias botánicas, cuidados, ubicaciones, recordatorios e historial.

La app móvil no se conectará directamente a MySQL. Todas las operaciones se realizarán a través de la API, lo que mejora la seguridad y evita exponer credenciales de base de datos en el dispositivo del usuario.

2.3 MODELO DE FUNCIONAMIENTO

El modelo de funcionamiento de Véridis se basa en una aplicación móvil gratuita conectada a una base de datos remota mediante una API REST.

El usuario puede crear una cuenta, iniciar sesión y registrar sus plantas personales. Cada planta registrada por el usuario está asociada a una planta base almacenada en el sistema. Esta separación permite reutilizar información común sobre la especie, la familia botánica, los cuidados recomendados y las recomendaciones generales.

A partir de esta información, el usuario puede consultar sus plantas desde el móvil, revisar sus datos principales, registrar acciones de cuidado y recibir recordatorios relacionados con tareas como riego, abonado, poda o trasplante.

Cuando el usuario realiza una acción en la aplicación, la app envía una petición a la API REST. El backend procesa la solicitud, comprueba que los datos sean correctos y actualiza la información correspondiente en la base de datos MySQL. Después, la API devuelve una respuesta a la aplicación para mostrar el resultado al usuario.

Este modelo permite construir un sistema organizado, seguro y preparado para futuras ampliaciones, como la incorporación de nuevas especies, mejoras en las recomendaciones, estadísticas de cuidado o nuevas funcionalidades relacionadas con el seguimiento de las plantas.

2.4 DISEÑO DE LA BASE DE DATOS

La base de datos de Véridis se ha diseñado para almacenar de forma organizada toda la información necesaria para el funcionamiento de la aplicación. Al tratarse de un sistema que gestiona usuarios, plantas, cuidados, recordatorios, fotografías, síntomas y recomendaciones, se ha utilizado una base de datos relacional en **MySQL**.

El diseño de la base de datos está formado por **14 tablas principales**:

- USUARIO
- CONFIG_USUARIO
- LOCALIZACION
- UBICACION
- FAMILIA_PLANTA
- PLANTA

- PLANTA_USUARIO
- FOTO_PLANTA
- HISTORIAL_CUIDADO
- RECORDATORIO
- SINTOMA
- RECOMENDACION_SINTOMA
- CUESTIONARIO
- RESPUESTA_CUESTIONARIO

La Figura 1 muestra el esquema de la base de datos realizado con draw.io, donde se representan las tablas principales y sus relaciones.

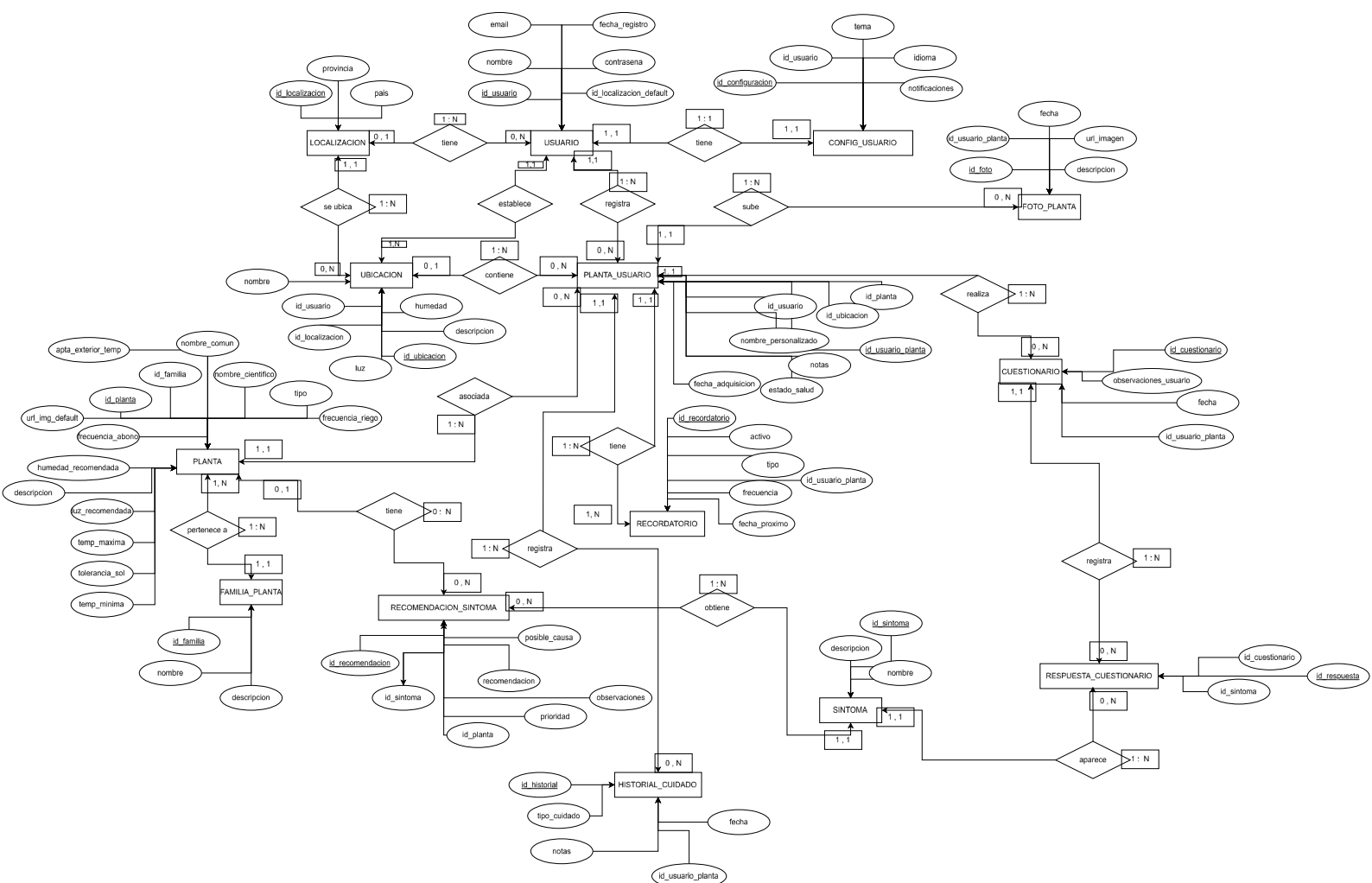


Figura 1 : Esquema entidad-relación de la base de datos de Véridis.

2.4.1 Descripción de las tablas

La tabla **USUARIO** almacena los datos principales de las personas registradas en la aplicación. Sirve como punto de partida para relacionar la información personal del usuario con sus plantas, configuraciones, ubicaciones y acciones dentro del sistema.

La tabla **CONFIG_USUARIO** guarda las preferencias asociadas a cada usuario. Esta tabla permite separar los datos de la cuenta de las opciones de configuración, como ajustes relacionados con notificaciones u otras preferencias de uso.

La tabla **LOCALIZACIÓN** almacena información relacionada con la localización geográfica. Esta información puede utilizarse para contextualizar datos externos, como la información meteorológica, y adaptar algunas recomendaciones al entorno del usuario.

La tabla **UBICACIÓN** registra los espacios donde el usuario coloca sus plantas, como salón, dormitorio, terraza o balcón. Esta tabla permite organizar mejor las plantas personales dentro de la aplicación.

La tabla **FAMILIA_PLANTA** almacena las familias botánicas. Su objetivo es clasificar las plantas de forma ordenada y aportar una estructura más clara al catálogo vegetal.

La tabla **PLANTA** contiene la información general de las plantas base del sistema. Esta tabla funciona como catálogo principal y recoge los datos comunes de cada especie o tipo de planta.

La tabla **PLANTA_USUARIO** almacena las plantas concretas registradas por cada usuario. Esta tabla permite diferenciar entre una planta base del catálogo y el ejemplar real que posee una persona. En ella se guarda la relación entre el usuario, la planta base y los datos personalizados de ese ejemplar.

La tabla **FOTO_PLANTA** almacena las imágenes asociadas a las plantas. Permite guardar fotografías de referencia o imágenes añadidas por el usuario para identificar mejor cada planta.

La tabla **HISTORIAL_CUIDADO** registra las acciones realizadas sobre una planta del usuario. En esta tabla se pueden almacenar cuidados como riegos, abonados, podas, trasplantes u observaciones importantes.

La tabla **RECORDATORIO** guarda las tareas programadas para el cuidado de las plantas. Permite avisar al usuario de acciones pendientes y controlar si una tarea ya ha sido realizada.

La tabla **SINTOMA** almacena los posibles síntomas o problemas que puede presentar una planta, como hojas amarillas, manchas, caída de hojas o falta de crecimiento.

La tabla **RECOMENDACION_SINTOMA** relaciona los síntomas con recomendaciones o posibles soluciones. Esta tabla permite ofrecer orientación al usuario cuando detecta algún problema en una planta.

La tabla **CUESTIONARIO** almacena la estructura de los cuestionarios utilizados dentro de la aplicación. Estos cuestionarios pueden servir para recopilar información sobre el estado de una planta o ayudar al usuario a identificar posibles problemas.

La tabla **RESPUESTA_CUESTIONARIO** guarda las respuestas dadas por el usuario en los cuestionarios. Esta tabla permite conservar la información introducida y relacionarla con el proceso de diagnóstico o recomendación correspondiente.

2.4.2 Relaciones principales

La base de datos se organiza mediante relaciones entre las distintas tablas para evitar duplicar información y mantener los datos correctamente conectados.

Un **usuario** puede tener asociada una configuración propia, varias ubicaciones y varias plantas personales. Esto permite que cada persona gestione su cuenta y su colección de plantas de forma independiente.

Una **familia de planta** puede estar relacionada con varias plantas. De esta forma, las especies del catálogo se organizan según su grupo botánico.

Una **planta base** puede aparecer en muchas plantas de usuario. Esto permite que varios usuarios registren ejemplares de una misma especie sin duplicar la información general de la planta.

Una **planta de usuario** puede tener fotografías, recordatorios e historial de cuidados. Esta relación permite hacer un seguimiento individual de cada ejemplar registrado.

Los **recordatorios** se asocian a plantas concretas del usuario y representan tareas pendientes o programadas. En cambio, el **historial de cuidado** registra acciones que ya se han realizado. Esta separación permite diferenciar entre lo que está planificado y lo que ya ha ocurrido.

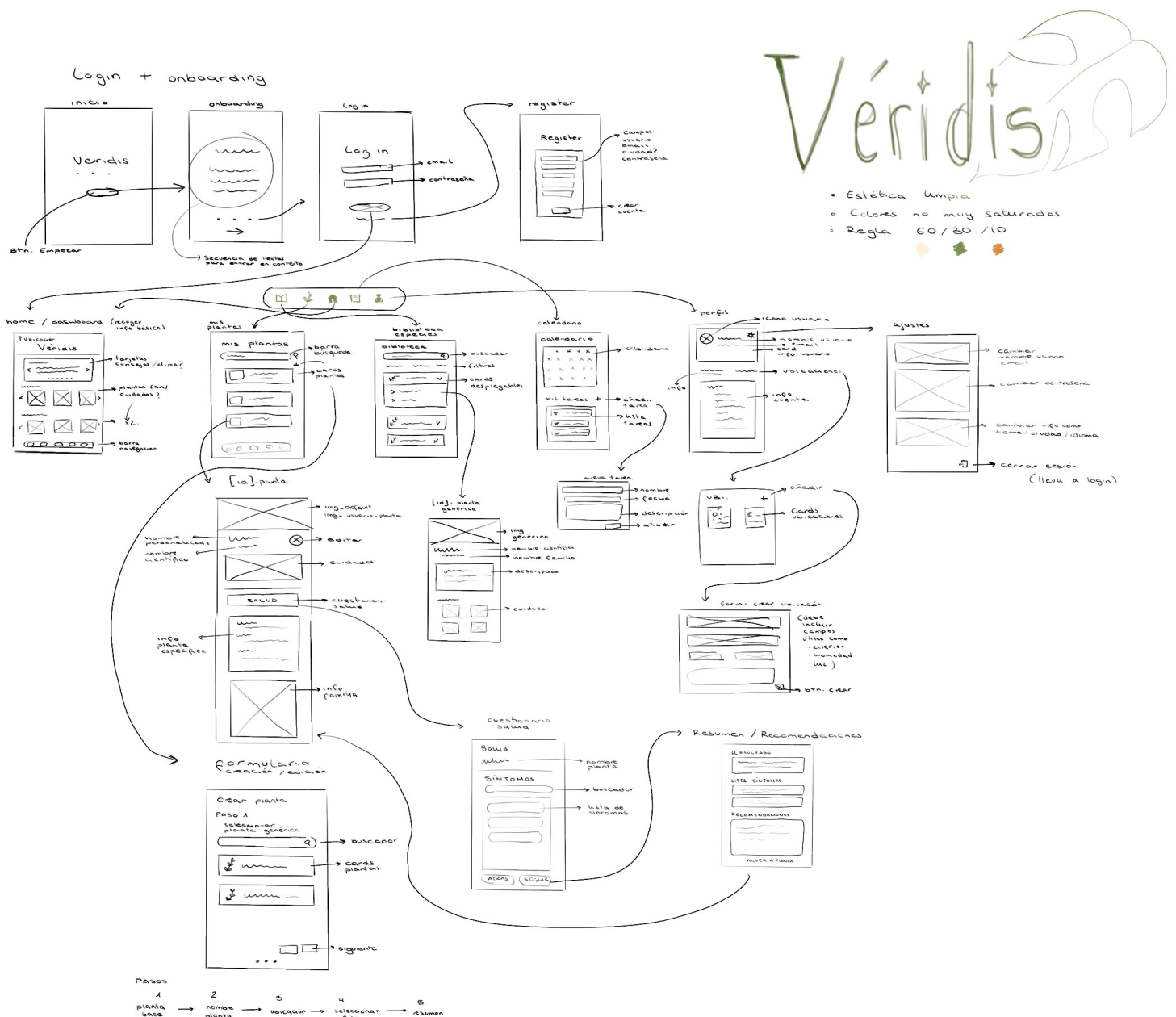
Los **síntomas** se relacionan con recomendaciones mediante la tabla **RECOMENDACION_SINTOMA**. Gracias a esta estructura, el sistema puede mostrar consejos o posibles soluciones según el problema detectado.

Los **cuestionarios** se relacionan con sus respuestas mediante la tabla **RESPUESTA_CUESTIONARIO**. Esto permite almacenar las respuestas del usuario y utilizarlas como apoyo para valorar el estado de una planta o mostrar recomendaciones.

En conjunto, este diseño permite mantener una base de datos organizada, evitar información repetida y facilitar futuras ampliaciones del sistema, como añadir nuevas plantas, nuevos síntomas, más recomendaciones, estadísticas de cuidado o mejoras en los cuestionarios.

La navegación de Véridis se ha diseñado para que el usuario pueda acceder de forma rápida a las funciones principales de la aplicación. Al tratarse de una app pensada para un uso frecuente, la estructura se organiza en pantallas claras, con accesos directos a las zonas más importantes: inicio, plantas, cuidados, calendario, recomendaciones y ajustes.

Figura 2 : Maa de navegación de Véridis.



2.5.1 Flujo inicial de acceso

Al abrir la aplicación, el usuario accede en primer lugar a una pantalla inicial o splash screen. Esta pantalla sirve como presentación visual de la app y da paso al flujo de acceso.

Después, se muestra una sección de onboarding en la que se introduce brevemente la finalidad de Véridis. En esta parte se explica que la aplicación permite registrar plantas, organizar sus cuidados, recibir recordatorios y consultar información útil para mantenerlas en buen estado.

Desde este punto, el usuario puede acceder a las pantallas de **inicio de sesión o registro**. Si ya tiene una cuenta, inicia sesión con sus credenciales. Si es un usuario nuevo, puede crear una cuenta introduciendo los datos necesarios.

2.5.2 Pantalla principal

Una vez iniciado sesión, el usuario accede a la pantalla principal de la aplicación. Esta pantalla funciona como dashboard y resume la información más relevante del usuario.

Desde la pantalla principal se pueden consultar las plantas registradas recientemente, los cuidados pendientes del día, avisos activos, recomendaciones generales y datos relacionados con el clima o la ubicación del usuario.

El objetivo de esta pantalla es que el usuario pueda ver rápidamente qué plantas necesitan atención sin tener que recorrer toda la aplicación.

2.5.3 Navegación principal

La aplicación se organiza mediante una navegación principal inferior, pensada para facilitar el acceso a las secciones más utilizadas. Esta navegación permite cambiar entre las áreas principales sin perder claridad.

Las secciones principales son:

- **Inicio** : muestra el resumen general de la actividad del usuario.
- **Mis plantas** : permite consultar y gestionar las plantas registradas.
- **Biblioteca** : acceso a la sección que recoge la información de las familias botánicas.
- **Calendario** : muestra tareas, recordatorios y acciones programadas.
- **Ajustes** : permite modificar datos del usuario, localización y preferencias.

2.5.4 Navegación en el apartado de plantas

El apartado **Mis plantas** permite al usuario consultar todas las plantas que tiene registradas. Desde este listado se puede acceder al detalle de cada planta personal.

En la pantalla de detalle se muestra información personalizada del ejemplar, como nombre, imagen, ubicación, estado de salud, notas, próximos cuidados e historial. Desde esta pantalla también se pueden realizar acciones como editar la planta, registrar un cuidado, consultar recordatorios asociados o revisar información relacionada con la especie.

2.5.5 Biblioteca botánica

El apartado de biblioteca reúne la información de todas las familias botánicas registradas en la base de datos, incluyendo así las plantas que van relacionadas a estas. El usuario puede entrar a una pantalla de detalle dentro de cada planta para ver su descripción y sus cuidados.

Este apartado está pensado para que la aplicación también ofrezca un apartado de educación e información sobre las plantas.

2.5.6 Cuidados, calendario y recordatorios

La sección de cuidados permite consultar las tareas pendientes relacionadas con las plantas del usuario. Estas tareas pueden incluir riego, abonado, poda, trasplante u otras acciones de mantenimiento.

El calendario permite visualizar los días con tareas programadas y consultar qué cuidados corresponden a una fecha concreta.

2.5.7 Síntomas, cuestionarios y recomendaciones

Véridis también incluye una parte orientada a ayudar al usuario cuando una planta presenta algún problema. Para ello, se contempla una navegación basada en síntomas, cuestionarios y recomendaciones.

El usuario puede seleccionar síntomas observados en una planta o responder a un cuestionario sencillo. A partir de esa información, la aplicación puede mostrar recomendaciones relacionadas con posibles causas o cuidados que conviene revisar.

Este apartado no se plantea como un diagnóstico médico o automático por inteligencia artificial, sino como una ayuda orientativa basada en información estructurada.

2.5.8 Ubicaciones

La aplicación permite organizar las plantas según la ubicación en la que se encuentren. El usuario puede crear ubicaciones como salón, dormitorio, terraza o balcón, y asignar plantas a cada una de ellas.

Esta función ayuda a mantener la colección de plantas más ordenada y permite contextualizar mejor algunos cuidados, especialmente cuando una planta se encuentra en interior o exterior.

2.5.9 Ajustes y perfil de usuario

La sección de ajustes permite gestionar información relacionada con la cuenta del usuario y sus preferencias. Desde esta pantalla se pueden modificar datos básicos como nombre, correo electrónico, contraseña, ciudad principal o preferencias de notificaciones.

La ciudad principal se utiliza como referencia para funciones relacionadas con el clima, recomendaciones y contexto general de la aplicación.

También desde ajustes se puede cerrar sesión, volviendo a la pantalla de acceso.

3. FASE DE PRUEBAS

La fase de pruebas tiene como finalidad comprobar que la aplicación Véridis funciona correctamente y que las funcionalidades implementadas cumplen los objetivos definidos en el proyecto. Para ello, se realizará una batería de pruebas orientada a detectar errores lógicos, fallos de navegación, problemas en la comunicación con el backend, errores en la base de datos y posibles mejoras en la experiencia de usuario.

Las pruebas se centrarán en verificar el funcionamiento de los módulos principales del sistema: autenticación, gestión de plantas, consulta de información botánica, recordatorios, historial de cuidados, navegación entre pantallas, conexión con la API REST y operaciones sobre la base de datos MySQL.

El objetivo principal de esta fase no es únicamente comprobar que la aplicación no presenta errores visibles, sino confirmar que el sistema responde de forma coherente ante distintos casos de uso, incluyendo acciones correctas, datos incompletos, errores del usuario y situaciones en las que no existan datos disponibles.

3.1 OBJETIVOS DE LAS PRUEBAS

Los objetivos principales de la fase de pruebas son los siguientes:

- Comprobar que la aplicación permite al usuario navegar correctamente entre las distintas pantallas.
- Verificar que el registro e inicio de sesión funcionan de forma adecuada.
- Comprobar que las operaciones de creación, lectura, modificación y eliminación de plantas se realizan correctamente.
- Validar que la aplicación se comunica correctamente con la API.
- Confirmar que los datos se almacenan y recuperan correctamente desde la base de datos MySQL.
- Detectar posibles errores en formularios, validaciones y mensajes mostrados al usuario.
- Comprobar que los recordatorios y el historial de cuidados funcionan de forma coherente.
- Evaluar la usabilidad general de la aplicación en un dispositivo móvil.
- Identificar posibles mejoras antes de la entrega final del prototipo.

3.2 TIPOS DE PRUEBAS REALIZADAS

Para comprobar el correcto funcionamiento del sistema se plantean varios tipos de pruebas:

Pruebas funcionales

Estas pruebas verifican que cada funcionalidad principal de la aplicación cumple con el comportamiento esperado. Se comprueba, por ejemplo, que el usuario pueda iniciar sesión, registrar una planta, consultar sus datos, modificar información o crear recordatorios.

Pruebas de validación

Estas pruebas se centran en comprobar que los formularios controlan correctamente los datos introducidos por el usuario. Se revisan casos como campos vacíos, formatos incorrectos, contraseñas no válidas o datos obligatorios sin completar.

Pruebas de navegación

Estas pruebas permiten comprobar que el usuario puede desplazarse correctamente por las pantallas de la aplicación. Se revisa que los botones, pestañas y enlaces internos dirijan a la pantalla correspondiente y que no existan rutas rotas.

Pruebas de base de datos

Estas pruebas verifican que la información se guarda, consulta, modifica y elimina correctamente en la base de datos. También se comprueba que las relaciones entre tablas funcionan de forma adecuada, especialmente entre usuarios, plantas base, plantas del usuario, ubicaciones, recordatorios e historial de cuidados.

Pruebas de API REST

Estas pruebas comprueban que los endpoints del backend responden correctamente a las peticiones realizadas desde la aplicación móvil. Se revisan métodos como GET, POST, PUT/PATCH y DELETE, así como los códigos de respuesta y los datos devueltos en formato JSON.

Pruebas de usabilidad

Estas pruebas valoran si la aplicación resulta clara, intuitiva y cómoda de utilizar. Se revisa la comprensión de los formularios, la visibilidad de los botones, la organización de la información y la facilidad para realizar las acciones principales.

3.3 BATERÍA DE PRUEBAS

Código	Área	Prueba	Resultado esperado
P01	Inicio de la aplicación	Abrir la aplicación desde el dispositivo o emulador	La aplicación se inicia correctamente y muestra la pantalla inicial sin errores
P02	Navegación	Acceder desde la pantalla inicial a la pantalla de inicio de sesión	La navegación se realiza correctamente

Código	Área	Prueba	Resultado esperado
P03	Registro	Registrar un usuario con datos válidos	El usuario se crea correctamente y queda almacenado en la base de datos
P04	Registro	Intentar registrar un usuario con campos obligatorios vacíos	El sistema muestra un mensaje de error e impide continuar
P05	Registro	Intentar registrar un usuario con un correo ya existente	El sistema impide duplicar el usuario
P06	Inicio de sesión	Iniciar sesión con credenciales correctas	El usuario accede correctamente a la pantalla principal
P07	Inicio de sesión	Iniciar sesión con credenciales incorrectas	El sistema rechaza el acceso y muestra un aviso
P08	Sesión de usuario	Cerrar sesión desde la aplicación	La sesión se cierra y el usuario vuelve a la pantalla de acceso
P09	Dashboard	Cargar la pantalla principal del usuario	Se muestran correctamente las plantas, avisos o secciones principales
P10	Catálogo de plantas	Consultar la lista de plantas base	La aplicación muestra las plantas disponibles desde la base de datos
P11	Catálogo de plantas	Acceder al detalle de una planta base	Se muestra la información de la especie, cuidados y consejos asociados
P12	Plantas del usuario	Registrar una nueva planta del usuario	La planta se guarda correctamente asociada al usuario
P13	Plantas del usuario	Registrar una planta sin campos obligatorios	El sistema impide guardar la planta incompleta
P14	Plantas del usuario	Consultar el listado de plantas personales	Se muestran únicamente las plantas asociadas al usuario activo
P15	Plantas del usuario	Acceder al detalle de una planta personal	Se muestra la información personalizada de la planta seleccionada
P16	Plantas del usuario	Modificar los datos de una planta personal	Los cambios se guardan correctamente en la base de datos

Código	Área	Prueba	Resultado esperado
P17	Plantas del usuario	Eliminar o desactivar una planta personal	La planta deja de mostrarse en el listado principal
P18	Ubicaciones	Crear una ubicación para organizar plantas	La ubicación se guarda correctamente
P19	Ubicaciones	Asignar una planta a una ubicación	La planta queda asociada a la ubicación seleccionada
P20	Historial de cuidados	Registrar una acción de cuidado realizada	El evento queda guardado en el historial de la planta
P21	Historial de cuidados	Consultar el historial de una planta	Se muestran las acciones realizadas ordenadas correctamente
P22	Recordatorios	Crear un recordatorio de riego, abonado u otra tarea	El recordatorio se guarda asociado a la planta correspondiente
P23	Recordatorios	Marcar un recordatorio como completado	El recordatorio cambia de estado y registra la fecha de finalización
P24	Recordatorios	Consultar recordatorios pendientes	Se muestran las tareas activas del usuario
P25	Notificaciones	Programar una notificación local	La notificación queda preparada para avisar al usuario
P26	API REST	Realizar una petición GET desde la app	El backend devuelve los datos solicitados en formato JSON
P27	API REST	Realizar una petición POST desde la app	El backend recibe los datos y crea el registro correspondiente
P28	API REST	Realizar una petición PUT/PATCH desde la app	El backend actualiza correctamente el registro indicado
P29	API REST	Realizar una petición DELETE desde la app	El backend elimina o desactiva correctamente el registro indicado
P30	Base de datos	Comprobar las relaciones entre tablas	Las claves foráneas mantienen la coherencia de los datos

Código	Área	Prueba	Resultado esperado
P31	Base de datos	Intentar acceder a datos de otro usuario	El sistema no muestra información que no pertenezca al usuario activo
P32	Clima	Consultar información meteorológica según ubicación	La aplicación muestra datos climáticos relacionados con la ubicación del usuario
P33	Interfaz	Revisar formularios, botones y textos	La interfaz se muestra de forma clara y sin elementos descolocados
P34	Usabilidad	Realizar el flujo completo de uso básico	El usuario puede registrarse, añadir una planta, crear un recordatorio y registrar un cuidado sin dificultad
P35	Control de errores	Simular un fallo de conexión con el backend	La aplicación muestra un mensaje de error comprensible
P36	Rendimiento	Cargar pantallas principales con varios registros	La aplicación mantiene una carga fluida y sin bloqueos importantes

3.4 CRITERIOS DE ACEPTACIÓN

Se considerará que una funcionalidad supera la prueba cuando cumpla las siguientes condiciones:

- La acción se realiza sin errores visibles para el usuario.
- Los datos introducidos se guardan correctamente en la base de datos.
- La aplicación muestra mensajes adecuados cuando ocurre un error.
- La navegación lleva a la pantalla esperada.
- La información mostrada corresponde al usuario activo.
- La respuesta de la API es coherente con la operación realizada.
- La interfaz mantiene una visualización correcta en el dispositivo de prueba.

En caso de que alguna prueba no se supere, se revisará el origen del fallo, diferenciando si se trata de un error de interfaz, lógica de aplicación, comunicación con el backend, validación de datos o base de datos.

3.5 RESULTADOS DE LA FASE DE PRUEBAS

Código	Área	Resultado obtenido
P01	Inicio de la aplicación	La aplicación se abrió correctamente y mostró la pantalla inicial sin errores.
P02	Navegación	La navegación hacia la pantalla de inicio de sesión se realizó correctamente.
P03	Registro	El usuario se creó correctamente y quedó registrado en la base de datos.
P04	Registro	El sistema mostró un aviso e impidió continuar con campos vacíos.
P05	Registro	El sistema detectó el correo duplicado e impidió crear otro usuario.
P06	Inicio de sesión	El usuario accedió correctamente a la pantalla principal.
P07	Inicio de sesión	El sistema rechazó el acceso y mostró un mensaje de error.
P08	Sesión de usuario	La sesión se cerró correctamente y volvió a la pantalla de acceso.

Código	Área	Resultado obtenido
P09	Dashboard	La pantalla principal cargó correctamente las secciones principales.
P10	Catálogo de plantas	La aplicación mostró correctamente las plantas disponibles.
P11	Catálogo de plantas	Se mostró correctamente la información de la planta seleccionada.
P12	Plantas del usuario	La planta se guardó correctamente asociada al usuario activo.
P13	Plantas del usuario	El sistema impidió guardar la planta incompleta y mostró un aviso.
P14	Plantas del usuario	Se mostraron únicamente las plantas del usuario activo.
P15	Plantas del usuario	Se mostró correctamente la información personalizada de la planta.
P16	Plantas del usuario	Los cambios realizados se guardaron correctamente.
P17	Plantas del usuario	La planta dejó de mostrarse en el listado principal.
P18	Ubicaciones	La ubicación se creó y guardó correctamente.
P19	Ubicaciones	La planta quedó asociada correctamente a la ubicación seleccionada.
P20	Historial de cuidados	El cuidado quedó registrado correctamente en el historial.
P21	Historial de cuidados	El historial se mostró correctamente y ordenado.
P22	Recordatorios	El recordatorio se guardó correctamente asociado a la planta.
P23	Recordatorios	El recordatorio cambió a completado y registró la fecha correspondiente.
P24	Recordatorios	Se mostraron correctamente los recordatorios pendientes.
P25	Notificaciones	La notificación quedó programada correctamente.
P26	API REST	El backend devolvió los datos solicitados en formato JSON.
P27	API REST	El backend recibió los datos y creó el registro correctamente.
P28	API REST	El backend actualizó correctamente el registro indicado.
P29	API REST	El backend eliminó o desactivó correctamente el registro indicado.

Código	Área	Resultado obtenido
P30	Base de datos	Las relaciones entre tablas mantuvieron la coherencia de los datos.
P31	Base de datos	El sistema mostró únicamente los datos del usuario activo.
P32	Clima	La aplicación mostró correctamente los datos climáticos según la ubicación.
P33	Interfaz	Los formularios, botones y textos se visualizaron correctamente.
P34	Usabilidad	El flujo completo se realizó correctamente y sin bloqueos.
P35	Control de errores	La aplicación mostró un mensaje de error claro y comprensible.
P36	Rendimiento	Las pantallas principales cargaron de forma fluida y sin bloqueos importantes.

Se ha elaborado una batería de pruebas funcionales para comprobar el comportamiento general de Véridis. Estas pruebas cubren las áreas principales de la aplicación, como registro, inicio de sesión, navegación, gestión de plantas, ubicaciones, historial de cuidados, recordatorios, conexión con la API REST, base de datos, interfaz y rendimiento. En las pruebas verificadas, el resultado ha sido correcto, cumpliéndose el comportamiento esperado en cada caso.

4. DOCUMENTACIÓN DE LA APLICACIÓN

4.1 INTRODUCCIÓN A LA APLICACIÓN

Véridis es una aplicación móvil orientada al cuidado y seguimiento de plantas domésticas. Su finalidad es ofrecer al usuario una herramienta sencilla para organizar sus plantas, consultar información útil sobre ellas y controlar las tareas de mantenimiento necesarias para conservarlas en buen estado.

La aplicación está pensada para un uso cotidiano desde el teléfono móvil. Por ello, su funcionamiento se centra en acciones rápidas y frecuentes, como consultar las plantas registradas, revisar recordatorios, añadir una nueva planta, registrar un cuidado realizado o comprobar información relacionada con una especie concreta.

El usuario accede a la aplicación mediante un sistema de registro e inicio de sesión. Una vez dentro, puede gestionar su colección personal de plantas. Cada planta registrada puede estar asociada a una planta base del sistema, lo que permite aprovechar información general sobre la especie, como su familia botánica, sus características principales y sus cuidados recomendados.

Véridis también permite organizar las plantas por ubicaciones, añadir imágenes, consultar el historial de cuidados y recibir avisos relacionados con tareas pendientes. De esta forma, la aplicación no se limita a mostrar información, sino que ayuda al usuario a mantener un seguimiento continuado de sus plantas.

La aplicación se apoya en una base de datos remota conectada mediante una API REST. Gracias a esta estructura, la información del usuario queda almacenada de forma persistente y puede gestionarse de manera segura desde la aplicación móvil.

En conjunto, Véridis ofrece una experiencia clara y ordenada, orientada a personas que tienen plantas en casa y quieren cuidarlas mejor sin depender únicamente de la memoria o de anotaciones manuales.

4.2 MANUAL DE INSTALACIÓN

En este apartado se explica el proceso necesario para instalar y ejecutar Véridis en un entorno de desarrollo. La aplicación está formada por tres partes principales: la aplicación móvil, la API REST y la base de datos MySQL.

4.2.1 Requisitos previos

Para poder instalar y ejecutar el proyecto será necesario disponer de los siguientes elementos:

- Un ordenador con Windows, Linux o macOS.
- Visual Studio Code o un editor de código similar.
- Node.js y npm instalados.
- Expo CLI o el uso de Expo mediante npx.
- Java Development Kit compatible con Spring Boot.
- Maven o el gestor de dependencias incluido en el proyecto.
- MySQL o acceso a una base de datos MySQL remota.
- Android Studio con un emulador configurado o un dispositivo móvil físico.
- Git, en caso de descargar el proyecto desde un repositorio.

4.2.2 Instalación de la base de datos

En primer lugar, se debe disponer de una base de datos MySQL creada para el proyecto. Esta base de datos almacenará toda la información persistente de la aplicación, como usuarios, plantas, ubicaciones, recordatorios, historial de cuidados, síntomas y recomendaciones.

Si la base de datos se instala en local, se deberá crear desde MySQL Workbench, phpMyAdmin o consola. En caso de utilizar una base de datos remota, será necesario contar con los datos de conexión: servidor, puerto, nombre de la base de datos, usuario y contraseña.

4.2.3 Instalación y configuración de la API REST

La API REST se desarrolla con Spring Boot y JPA. Para instalarla, se debe abrir la carpeta del backend en el entorno de desarrollo y comprobar que las dependencias del proyecto están correctamente configuradas.

A continuación, se deben configurar los datos de conexión a MySQL en el archivo de configuración correspondiente, normalmente `application.properties`

Una vez configurada la conexión, la API puede ejecutarse desde el entorno de desarrollo o mediante Maven. Si el proyecto está correctamente configurado, el servidor se iniciará y quedará preparado para recibir peticiones desde la aplicación móvil.

La API será la encargada de gestionar las operaciones relacionadas con usuarios, plantas, ubicaciones, recordatorios, historial y el resto de información almacenada en la base de datos.

4.2.4 Instalación de la aplicación móvil

La aplicación móvil se desarrolla con React Native y Expo. Para instalarla, se debe abrir la carpeta del frontend y ejecutar la instalación de dependencias:

```
npm install
```

Después, se debe configurar la URL de la API REST para que la aplicación pueda comunicarse con el backend. Esta URL debe apuntar al servidor donde se esté ejecutando la API, ya sea en local o en remoto.

```
http://localhost:8080
```

Si se utiliza un emulador o un dispositivo físico, puede ser necesario sustituir `localhost` por la dirección IP del ordenador o del servidor donde se esté ejecutando la API.

4.2.5 Ejecución de la aplicación móvil

Una vez instaladas las dependencias, la aplicación puede iniciar con expo mediante el siguiente comando:

```
npx expo start
```

Este comando abrirá el panel de Expo y permitirá ejecutar la aplicación en un emulador Android o en un dispositivo físico mediante Expo Go.

Si se utiliza un dispositivo físico, el móvil y el ordenador deberán estar conectados a la misma red para que la aplicación pueda comunicarse correctamente con la API local.

4.2.6 Comprobación de funcionamiento

Para comprobar que la instalación se ha realizado correctamente, se deben revisar los siguientes puntos:

- La base de datos está activa y accesible.
- La API REST se inicia sin errores.
- La aplicación móvil se abre correctamente desde Expo.
- La app puede comunicarse con el backend.
- El usuario puede registrarse o iniciar sesión.
- La aplicación puede consultar y guardar datos en la base de datos.

Si alguna de estas partes falla, se deberá revisar la configuración de conexión, los puertos utilizados, la URL de la API o el estado de la base de datos.

4.3 MANUAL DE ADMINISTRACIÓN

Véridis no dispone de una interfaz gráfica específica para el administrador. Esta decisión se ha tomado de forma intencionada, ya que la aplicación está planteada para funcionar mediante versiones controladas y actualizaciones periódicas de contenido.

El sistema no está pensado para que un administrador modifique datos constantemente desde un panel interno, sino para que el contenido principal de la aplicación se revise, prepare y publique de manera ordenada. De esta forma, las nuevas plantas, síntomas, recomendaciones, cuestionarios o mejoras de contenido se incorporan en bloques de actualización, evitando cambios improvisados o poco controlados dentro de la base de datos.

Esta forma de administración permite mantener un mayor control sobre la calidad de la información que se muestra al usuario. Al tratarse de una aplicación relacionada con el cuidado de plantas, es importante que los datos añadidos sean coherentes, estén revisados y mantengan una estructura común. Por este motivo, se ha considerado más adecuado gestionar el contenido mediante actualizaciones planificadas que mediante un panel de administración abierto a cambios continuos.

4.4.1 Justificación de la ausencia de la interfaz de administración

Aunque muchas aplicaciones cuentan con una interfaz de administración, en el caso de Véridis no se ha considerado imprescindible para el prototipo desarrollado. La aplicación está enfocada principalmente al usuario final, que será quien registre sus plantas, consulte información, reciba recordatorios y gestione sus cuidados.

La administración del sistema se centra en mantener y ampliar la información base de la aplicación. Esta información incluye plantas, familias botánicas, síntomas, recomendaciones y cuestionarios. Al ser datos que no necesitan modificarse diariamente desde la propia aplicación, se ha optado por una gestión más controlada mediante versiones.

Esta decisión permite:

- Evitar modificaciones accidentales en datos importantes.
- Mantener controladas las versiones de la aplicación.
- Revisar el contenido antes de publicarlo.
- Asegurar que las plantas, síntomas y recomendaciones siguen una estructura coherente.
- Reducir la complejidad del sistema en esta fase del proyecto.
- Priorizar el desarrollo de las funcionalidades principales del usuario final.

Por tanto, la ausencia de una interfaz gráfica de administración no significa que el sistema no pueda administrarse, sino que la administración se realiza de forma técnica y planificada.

4.4.2 Tareas de administración

Las principales tareas de administración del sistema son las siguientes:

Gestión de plantas base

Consiste en añadir, revisar o modificar la información general de las plantas disponibles en el sistema. Esta información puede incluir nombre común, nombre científico, familia botánica, descripción, características principales o imagen de referencia.

Gestión de familias botánicas

Permite organizar las plantas dentro de grupos o familias. Esta clasificación ayuda a mantener el catálogo más ordenado y facilita la consulta de información por parte del usuario.

Gestión de síntomas y recomendaciones

Incluye la revisión de los síntomas que puede presentar una planta y las recomendaciones asociadas. Esta parte debe mantenerse especialmente controlada, ya que influye en los consejos que recibe el usuario.

Gestión de cuestionarios

Consiste en revisar o ampliar las preguntas y respuestas utilizadas para orientar al usuario cuando una planta presenta algún problema.

Revisión de usuarios y datos asociados

Aunque el usuario gestiona sus propios datos desde la aplicación, a nivel técnico se puede comprobar que la información se almacena correctamente en la base de datos y que las relaciones entre usuarios, plantas, ubicaciones, recordatorios e historial se mantienen de forma coherente.

Mantenimiento de la base de datos

Incluye tareas como comprobar la integridad de los datos, revisar relaciones entre tablas, realizar copias de seguridad y verificar que no existan registros inconsistentes.

4.4.3 Copias de seguridad

Para evitar la pérdida de información, se recomienda realizar copias de seguridad periódicas de la base de datos. Estas copias deben incluir tanto la estructura de las tablas como los datos almacenados.

Las copias de seguridad son especialmente importantes antes de aplicar cambios importantes, añadir nuevos bloques de contenido o modificar relaciones entre tablas.

En caso de error, una copia de seguridad permitiría restaurar el sistema a un estado anterior estable.

5. CONCLUSIONES FINALES

El desarrollo de Véridis ha permitido crear una aplicación móvil orientada al cuidado y seguimiento de plantas domésticas. El proyecto cumple con el objetivo principal planteado, ya que ofrece una solución organizada para registrar plantas, consultar información, gestionar recordatorios y llevar un control de los cuidados realizados.

La aplicación permite centralizar en un mismo sistema información que normalmente el usuario tendría dispersa en notas, calendarios o recordatorios manuales. Además, la separación entre plantas base y plantas del usuario facilita reutilizar datos generales sobre cada especie y, al mismo tiempo, permitir que cada persona gestione sus propios ejemplares de forma personalizada.



A nivel técnico, el proyecto cumple con una arquitectura adecuada para una aplicación multiplataforma. Se ha desarrollado una aplicación móvil conectada a una API REST y a una base de datos MySQL, lo que permite separar la interfaz, la lógica del servidor y el almacenamiento de datos. Esta estructura facilita el mantenimiento del sistema y permite ampliarlo en el futuro de forma más ordenada.

También se ha trabajado el diseño de la interfaz, la navegación entre pantallas y la organización de la información para que la aplicación resulte clara y fácil de utilizar. Las pruebas realizadas han permitido comprobar el funcionamiento general del sistema y detectar posibles mejoras en la experiencia de usuario.

En conjunto, Véridis cumple los objetivos fijados para el proyecto, ya que proporciona una herramienta útil, funcional y coherente con las necesidades de usuarios que quieren cuidar mejor sus plantas domésticas.

5.1 PROPUESTAS DE MODIFICACIÓN O AMPLIACIONES FUTURAS

Como mejora futura, se plantea traducir la aplicación al inglés. Esta ampliación permitiría llegar a un público más amplio y facilitaría el uso de Véridis por parte de usuarios que no hablen español. Para ello, sería necesario adaptar los textos de la interfaz, mensajes de error, formularios, nombres de secciones y contenido informativo de la aplicación.

Otra ampliación importante sería mejorar el sistema de revisión de salud de las plantas. Actualmente, el sistema puede apoyarse en síntomas, cuestionarios y recomendaciones, pero en una versión futura podría incorporarse una función de detección mediante imagen. Esta mejora permitiría que el usuario subiera una fotografía de su planta y que un sistema basado en inteligencia artificial analizara posibles síntomas, como manchas, hojas amarillas, sequedad, plagas o signos de deterioro.

Esta función no se ha incluido en la versión actual porque habría aumentado considerablemente la complejidad del proyecto. Sin embargo, se considera una mejora interesante para futuras versiones, ya que podría hacer que las recomendaciones fueran más precisas y adaptadas al estado real de cada planta.

También podrían incorporarse otras mejoras, como estadísticas de cuidados, más especies en el catálogo, nuevas recomendaciones, personalización avanzada de recordatorios y un panel de administración en caso de que el volumen de contenido creciera y fuese necesario gestionarlo desde una interfaz específica.

Estas ampliaciones permitirían hacer evolucionar Véridis hacia una aplicación más completa, manteniendo la base ya desarrollada y respetando la estructura organizada del proyecto.

6. BIBLIOGRAFÍA

[React Native. Documentación oficial.](#)

[Expo. Documentación oficial.](#)

[Expo Notifications. Documentación oficial.](#)

[Expo Location. Documentación oficial.](#)

[Expo ImagePicker. Documentación oficial.](#)

[Open-Meteo. Documentación oficial.](#)

[Expo Router. Documentación oficial.](#)

[React Native Calendars. Documentación oficial.](#)

Apuntes del módulo de Acceso a datos.

Apuntes del módulo de Programación Multimedia y Dispositivos Móviles.

Apuntes del módulo de Desarrollo de Interfaces.